

HEALTHCARE + FITNESS AGENT

Feature Development Plan & Engineering Guide

Version 2.0 · May 2025
6 Feature Modules · 12 Sprints · 24 Weeks

F1 Google OAuth F2 Stripe Payments F3 Trend Charts
F4 Medication Tracking F5 Longitudinal Analysis F6 Condition-Specific Plans

1 Executive Overview

This document is the definitive engineering guide for extending the Healthcare + Fitness Agent from its Phase 1 foundation to a fully-featured, commercially viable health platform. It covers six feature modules, each with detailed implementation instructions, database schema changes, API contracts, frontend components, and testing requirements.

1.1 Feature Summary

#	Feature	Phase	Sprint(s)	Effort	LLM Cost
F1	Google OAuth + JWT Auth	1	S1	1 week	None
F2	Stripe Payment Integration	1	S2–S3	2 weeks	None
F3	Trend Charts & Time-Series	1	S4	1 week	Low
F4	Medication Tracking	2	S5–S6	2 weeks	Medium
F5	Multi-Report Longitudinal Analysis	2	S7–S9	3 weeks	High
F6	Condition-Specific Plans	2	S10–S12	3 weeks	High

1.2 Technology Stack (unchanged)

- Backend: FastAPI · LangGraph · SQLAlchemy · FAISS · Azure OpenAI (GPT-4o)
- Frontend: React 18 · Vite · TypeScript · Tailwind CSS · Zustand · Recharts
- Infrastructure: Docker Compose (Phase 1) · Azure Container Instances (Phase 2)
- New additions: Authlib (OAuth) · Stripe SDK · APScheduler · ReportLab

1.3 Architecture Impact Diagram

The diagram below shows all new components (shaded) alongside existing architecture:



risk · disease · fitness	+ condition_plan_agent + med_interaction_agent
Storage SQLite: + subscriptions · medications · goals [NEW] FAISS: unchanged (temporal metadata added to chunks) External: + Google OAuth · Stripe API	

2 Prerequisites & Environment Setup

2.1 New Dependencies

Add the following to `requirements.txt`:

Package	Version	Purpose
<code>authlib</code>	1.3.0	Google OAuth 2.0 / OIDC flow
<code>itsdangerous</code>	2.2.0	Session signing for OAuth state
<code>starlette</code>	0.37.2	SessionMiddleware (OAuth callback)
<code>stripe</code>	9.6.0	Stripe Checkout, Webhooks, Portal
<code>APScheduler</code>	3.10.4	Cron jobs (medication reminders)
<code>reportlab</code>	4.1.0	PDF export of health reports
<code>pandas</code>	2.2.0	Time-series aggregation for trends
<code>scipy</code>	1.13.0	Pearson correlation for longitudinal stats

2.2 External Services to Configure

Service	Where to Create	Config Keys Needed
Google Cloud Console	<code>console.cloud.google.com</code>	GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET
Stripe Dashboard	<code>dashboard.stripe.com</code>	STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET
Stripe Products	Dashboard > Products	PRICE_ID_STANDARD, PRICE_ID_PREMIUM
Azure OpenAI (existing)	<code>portal.azure.com</code>	Already configured in Phase 1

2.3 Updated .env Template

ENV	<pre># Existing Azure OpenAI keys remain unchanged GOOGLE_CLIENT_ID=xxx.apps.googleusercontent.com GOOGLE_CLIENT_SECRET=GOCSXP-xxxxxxxxxxxxxxxxx GOOGLE_REDIRECT_URI=http://localhost:8000/auth/callback STRIPE_SECRET_KEY=sk_test_xxxxxxxxxxxxxx STRIPE_WEBHOOK_SECRET=whsec_xxxxxxxxxxxxxx STRIPE_PRICE_STANDARD=price_xxxxxxxxxxxxxx STRIPE_PRICE_PREMIUM=price_xxxxxxxxxxxxxx APP_SECRET_KEY=<openssl rand -hex 32> # must be 32+ chars for session signing</pre>
-----	---

3 Database Schema Changes

All new tables must be added to `backend/db/models.py`. The existing four tables remain untouched. SQLAlchemy will auto-create all tables on startup via `Base.metadata.create_all()`.

3.1 New Tables Overview

Table	Feature	Primary Key	Key Relationships
subscriptions	F2 Stripe	id (auto)	user_id → users.user_id
medications	F4 Med Tracking	id (auto)	user_id → users.user_id
medication_logs	F4 Med Tracking	id (auto)	medication_id → medications.id
longitudinal_snapshots	F5 Analysis	id (auto)	user_id → users.user_id
condition_protocols	F6 Cond. Plans	id (auto)	user_id → users.user_id

3.2 subscriptions Table (F2)

SQL

```
class Subscription(Base):
    __tablename__ = "subscriptions"
    id = Column(Integer, primary_key=True)
    user_id = Column(String, ForeignKey("users.user_id"), unique=True, index=True)
    stripe_customer_id = Column(String, nullable=True)
    stripe_subscription_id = Column(String, nullable=True)
    plan = Column(String, default="free") # free | standard | premium
    status = Column(String, default="active") # active | cancelled | past_due
    upload_count = Column(Integer, default=0) # rolling monthly counter
    current_period_end = Column(DateTime, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)
```

3.3 medications & medication_logs Tables (F4)

SQL

```
class Medication(Base):
    __tablename__ = "medications"
    id = Column(Integer, primary_key=True)
    user_id = Column(String, ForeignKey("users.user_id"), index=True)
    name = Column(String, nullable=False)
    dosage = Column(String) # e.g. '10mg'
    frequency = Column(String) # e.g. 'twice daily'
    start_date = Column(Date)
    end_date = Column(Date, nullable=True)
    is_active = Column(Boolean, default=True)
    notes = Column(Text, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)
```

```
class MedicationLog(Base):
    __tablename__ = "medication_logs"
    id = Column(Integer, primary_key=True)
    medication_id = Column(Integer, ForeignKey("medications.id"), index=True)
    user_id = Column(String, ForeignKey("users.user_id"), index=True)
    taken_at = Column(DateTime, default=datetime.utcnow)
    status = Column(String) # taken | skipped | delayed
    notes = Column(String, nullable=True)
```

3.4 longitudinal_snapshots Table (F5)

SQL

```
class LongitudinalSnapshot(Base):
    __tablename__ = "longitudinal_snapshots"
    id = Column(Integer, primary_key=True)
    user_id = Column(String, ForeignKey("users.user_id"), index=True)
    snapshot_date = Column(Date, nullable=False)
    risk_score = Column(Integer)
    risk_level = Column(String)
    bmi = Column(Float, nullable=True)
    parameters_json = Column(Text) # JSON: {test_name: value, ...} for that date
    conditions_json = Column(Text) # JSON: [condition_name, ...] for that date
    file_id = Column(String) # which PDF triggered this snapshot
    created_at = Column(DateTime, default=datetime.utcnow)
```

4 Feature Modules — Implementation Guide

F1	Google OAuth 2.0 + JWT Authentication	1 Sprint	Medium
----	---------------------------------------	----------	--------

GOAL	Replace the current placeholder user_id='user-001' default with real Google identity. Every API endpoint is protected by JWT. User records in SQLite are automatically created on first login.
------	--

4.1.1 Backend — New Files

backend/routes/auth.py

CODE	<pre>from authlib.integrations.starlette_client import OAuth from starlette.middleware.sessions import SessionMiddleware from fastapi import APIRouter, Request, Depends from fastapi.responses import RedirectResponse router = APIRouter(prefix="/auth", tags=["auth"]) oauth = OAuth() oauth.register(name="google", client_id=settings.google_client_id, client_secret=settings.google_client_secret, server_metadata_url="https://accounts.google.com/.well-known/openid-configuration", client_kwargs={"scope": "openid email profile"}) @router.get("/login") async def login(request: Request): redirect_uri = settings.google_redirect_uri return await oauth.google.authorize_redirect(request, redirect_uri) @router.get("/callback") async def callback(request: Request, db: Session = Depends(get_db)): token = await oauth.google.authorize_access_token(request) user_info = token.get("userinfo") user = db.query(User).filter(User.email == user_info["email"]).first() if not user: user = User(user_id=user_info["sub"], email=user_info["email"]) db.add(user); db.commit() jwt_token = create_access_token({"sub": user.user_id, "email": user.email}) return RedirectResponse(f"http://localhost:3000/auth/success?token={jwt_token}")</pre>
------	--

backend/utils/jwt_utils.py

CODE	<pre>from jose import jwt from datetime import datetime, timedelta</pre>
------	--

```
def create_access_token(data: dict, expires_minutes: int = 1440) -> str:
    payload = {"**data", "exp": datetime.utcnow() + timedelta(minutes=expires_minutes)}
    return jwt.encode(payload, settings.app_secret_key, algorithm="HS256")

def decode_token(token: str) -> dict:
    return jwt.decode(token, settings.app_secret_key, algorithms=["HS256"])
```

backend/utils/auth_dep.py (FastAPI dependency)

CODE

```
from fastapi import Security, HTTPException
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

bearer = HTTPBearer()

def get_current_user(credentials: HTTPAuthorizationCredentials = Security(bearer)):
    try:
        return decode_token(credentials.credentials)
    except Exception:
        raise HTTPException(status_code=401, detail="Invalid or expired token")
```

4.1.2 Protect All Existing Routes

Add the dependency to every existing router. Example for `routes/upload.py`:

CODE

```
@router.post('/upload/{user_id}')
async def upload_pdf(user_id: str, file: UploadFile,
                    current_user = Depends(get_current_user),
                    db: Session = Depends(get_db)):
    if current_user['sub'] != user_id:
        raise HTTPException(403, 'Access denied')
    # ... existing logic unchanged
```

4.1.3 app.py Changes

- **Add:** SessionMiddleware before all other middleware
- **Add:** auth router → `app.include_router(auth_router)`

CODE

```
app.add_middleware(SessionMiddleware, secret_key=settings.app_secret_key)
```

4.1.4 Frontend Changes

- **New page:** `pages/LoginPage.tsx` — Google sign-in button → redirects to `/api/auth/login`
- **New page:** `pages/AuthSuccess.tsx` — extracts `?token=` from URL, stores in `localStorage`, redirects to `/dashboard`
- **API client:** Add Axios interceptor in `services/api.ts` to attach Bearer token to every request

- **Route guard:** Wrap all routes in `App.tsx` with a `PrivateRoute` component that checks `localStorage` for valid JWT
- **Zustand:** Add `currentUser: {sub, email} | null` field to global store; populate on `AuthSuccess`

4.1.5 Testing Checklist — F1

- Unit: `decode_token` returns correct payload; expired token raises `HTTPException`
- Unit: `get_current_user` with no Bearer header → 403
- Integration: GET `/auth/login` returns 302 redirect to `accounts.google.com`
- Integration: POST `/upload` with valid JWT + correct `user_id` → 200
- Integration: POST `/upload` with JWT `user_id` mismatch → 403
- E2E: Full Google OAuth flow in browser (use test Google account)

F2

Stripe Payment Integration

2 Sprints

Medium

GOAL

Gate premium features behind paid subscription tiers. Handle the full payment lifecycle: checkout → subscription active → renewal → cancellation → downgrade. Use Stripe-hosted pages for PCI compliance.

4.2.1 Subscription Tiers

Plan	Monthly Price	PDF Uploads/Month	AI Chat	Fitness Plans	PDF Export
Free	\$0	2	No	No	No
Standard	\$25	15	Yes	Yes	No
Premium	\$40	Unlimited	Yes	Yes	Yes

4.2.2 Backend — New Files

[backend/routes/billing.py](#)

CODE

```
import stripe
router = APIRouter(prefix='/billing', tags=['billing'])
stripe.api_key = settings.stripe_secret_key

PRICE_IDS = {
    'standard': settings.stripe_price_standard,
    'premium': settings.stripe_price_premium,
}
```

```

@router.post('/create-checkout-session')
async def checkout(plan: str, current_user=Depends(get_current_user)):
    session = stripe.checkout.Session.create(
        payment_method_types=['card'],
        mode='subscription',
        line_items=[{'price': PRICE_IDS[plan], 'quantity': 1}],

    success_url='http://localhost:3000/billing/success?session_id={CHECKOUT_SESSION_ID}',
    cancel_url='http://localhost:3000/billing/cancel',
    metadata={'user_id': current_user['sub']})
    return {'checkout_url': session.url}

@router.post('/webhook') # no JWT — Stripe calls this directly
async def webhook(request: Request, db=Depends(get_db)):
    payload = await request.body()
    sig = request.headers.get('stripe-signature')
    event = stripe.Webhook.construct_event(
        payload, sig, settings.stripe_webhook_secret)
    # Handle: checkout.session.completed → activate plan
    # Handle: customer.subscription.deleted → downgrade to free
    # Handle: invoice.payment_failed → mark past_due
    return {'status': 'ok'}

```

backend/utils/plan_guard.py (quota enforcement)

CODE

```

PDF_LIMITS = {'free': 2, 'standard': 15, 'premium': 999999}

def require_plan(min_plan: str):
    plans = ['free', 'standard', 'premium']
    def guard(current_user=Depends(get_current_user), db=Depends(get_db)):
        sub = db.query(Subscription).filter(
            Subscription.user_id == current_user['sub']).first()
        user_plan = sub.plan if sub else 'free'
        if plans.index(user_plan) < plans.index(min_plan):
            raise HTTPException(402, f'Upgrade to {min_plan} to use this feature')
        return Depends(guard)

def check_upload_quota(user_id: str, db: Session):
    sub = db.query(Subscription).filter(
        Subscription.user_id == user_id).first()
    plan = sub.plan if sub else 'free'
    limit = PDF_LIMITS[plan]
    if sub and sub.upload_count >= limit:
        raise HTTPException(403, f'Upload limit reached. Upgrade your plan.')
    # Increment counter after successful upload

```

4.2.3 Gating Existing Routes

Route	Minimum Plan	Guard Type
POST /upload/{user_id}	free (quota checked)	check_upload_quota()
POST /chat/	standard	require_plan('standard')
POST /fitness/plan	standard	require_plan('standard')
GET /insights/{user_id}/export.pdf	premium	require_plan('premium')

4.2.4 Frontend — New Pages

- **pages/BillingPage.tsx:** 3-card pricing grid (Free/Standard/Premium). 'Upgrade' button calls POST /api/billing/create-checkout-session. Stripe hosted checkout opens in same tab.
- **pages/BillingSuccess.tsx:** Post-checkout confirmation page. Refetches user subscription status from backend.
- **components/PlanBadge.tsx:** Small pill showing current plan (Free / Standard / Premium) displayed in the Navbar.
- **components/UpgradePrompt.tsx:** Modal shown when a user hits a paywalled feature. Contains plan comparison and Upgrade CTA.

4.2.5 Webhook Local Testing

CMD

Install Stripe CLI
brew install stripe/stripe-cli/stripe

Forward to local backend
stripe listen --forward-to localhost:8000/api/billing/webhook

Trigger test events
stripe trigger checkout.session.completed
stripe trigger customer.subscription.deleted

F3

Trend Charts & Time-Series Analytics

1 Sprint

Low

GOAL

Visualise how key health parameters change across multiple uploaded PDFs over time. Leverage the existing health_records.recorded_at column and new longitudinal_snapshots table. No additional LLM cost.

4.3.1 Backend — New Endpoint

[backend/routes/trends.py](#)

CODE

```

@router.get('/trends/{user_id}')
async def get_trends(
    user_id: str,
    parameters: str = Query('glucose,cholesterol,hemoglobin,creatinine'),
    days: int = Query(180),
    current_user=Depends(get_current_user),
    db=Depends(get_db)):

    param_list = [p.strip() for p in parameters.split(',')]
    since = datetime.utcnow() - timedelta(days=days)

    records = db.query(HealthRecord).filter(
        HealthRecord.user_id == user_id,
        HealthRecord.recorded_at >= since,
        HealthRecord.parameter.in_(param_list)
    ).order_by(HealthRecord.recorded_at.asc()).all()

    # Group by parameter name, return [{parameter, data:[{date,value}]}]
    from collections import defaultdict
    grouped = defaultdict(list)
    for r in records:
        grouped[r.parameter].append({
            'date': r.recorded_at.isoformat(),
            'value': float(r.value),
            'status': r.status
        })
    return {'user_id': user_id, 'series': [
        {'parameter': p, 'data': grouped[p]} for p in param_list if p in grouped
    ]}

```

4.3.2 Snapshot on PDF Upload

After every successful PDF ingestion in `services/pdf_service.py`, add a call to write a `LongitudinalSnapshot` row:

CODE

```

# At the END of process_pdf(), after DB inserts
snapshot = LongitudinalSnapshot(
    user_id=user_id,
    snapshot_date=date.today(),
    risk_score=computed_risk_score, # from risk_service
    risk_level=risk_level,
    bmi=bmi,
    parameters_json=json.dumps({r.parameter: r.value for r in new_records}),
    conditions_json=json.dumps(detected_conditions),
    file_id=file_id)
db.add(snapshot); db.commit()

```

4.3.3 Frontend — TrendsPage.tsx

- **Library:** Recharts is already in use for the risk bar. Extend with LineChart for parameter trends.
- **Parameter selector:** Multi-select checkbox group (glucose, cholesterol, hemoglobin, HbA1c, creatinine, triglycerides). Default: glucose + cholesterol.
- **Time range:** Dropdown: 30 days / 90 days / 180 days / All time.
- **Chart:** One LineChart per selected parameter. X-axis = date (formatted). Y-axis = value. Reference range shown as ReferenceLine (top/bottom bounds). Dot colour = Normal (green) / High (amber) / Critical (red).
- **Risk score trend:** AreaChart at the top showing risk_score across all snapshots.
- **No data state:** Prompt to upload at least 2 PDFs to see trends.

F4

Medication Tracking & Interaction Checking

2 Sprints

Medium

GOAL

Allow users to log their current medications, track daily adherence, and receive AI-powered drug interaction warnings based on their health conditions. Leverages the existing drug_knowledge/ static knowledge base.

4.4.1 Backend — New Endpoints

Method		Path	Purpose
POST		/medications/	Add a new medication
GET		/medications/{user_id}	List all active medications
PUT		/medications/{id}	Update medication (dosage/frequency/end_date)
DELETE		/medications/{id}	Deactivate a medication
POST		/medications/{id}/log	Log taken/skipped/delayed event
GET		/medications/{user_id}/adherence	Adherence % for last 30 days
GET		/medications/{user_id}/interactions	Check for drug interactions

backend/agents/med_interaction_agent.py (new Agent 6)

CODE

```
INTERACTION_PROMPT = ""
You are a clinical pharmacist AI assistant.
Given the patient's active medications and detected health conditions,
identify potential drug interactions, contraindications with their conditions,
and any parameters from their lab results that require monitoring.

Active medications: {medications}
Detected conditions: {conditions}
Flagged lab parameters: {flagged_params}
```

```
Return JSON: {
  "interactions": [{"drugs": [...], "severity": "mild|moderate|severe", "description": "..."}],
  "condition_contraindications": [{"medication": "...", "condition": "...", "warning": "..."}],
  "monitoring_required": [{"parameter": "...", "reason": "...", "frequency": "..."}]
}
NEVER advise stopping medication. Always recommend consulting a physician.
"
```

Drug knowledge base usage

INFO

The existing data/drug_knowledge/ directory contains static text files. On interaction check, load all .txt files from this directory into the prompt context. This grounds the LLM in curated knowledge rather than hallucination. Chunk and embed drug_knowledge files into a separate FAISS index on startup. Use similarity search to retrieve only relevant drug documents per query.

4.4.2 Frontend — MedicationsPage.tsx

- **Add medication form:** Name, dosage, frequency (once/twice/three times daily, as needed), start date, end date (optional), notes.
- **Active medications list:** Cards showing name, dosage, frequency, start date, adherence streak.
- **Daily log widget:** Today's medications listed with Taken / Skip / Delay buttons. Logs a medication_logs entry on click.
- **Adherence chart:** Recharts BarChart showing taken vs skipped per day for last 14 days.
- **Interactions panel:** GET /medications/{user_id}/interactions result displayed with colour-coded severity (mild=green, moderate=amber, severe=red). Always includes disclaimer.

F5

Multi-Report Longitudinal Analysis

3 Sprints

High

GOAL

Synthesise all historical PDFs into a coherent health narrative. Detect trajectory changes (improving/worsening/stable) per parameter, identify emerging conditions, and generate a clinician-ready longitudinal summary. This is the key differentiator versus competitors.

4.5.1 Backend — New Endpoint

GET /longitudinal/{user_id}

CODE

```
@router.get('/longitudinal/{user_id}')
async def longitudinal_analysis(user_id: str,
                               current_user=Depends(get_current_user),
                               db=Depends(get_db)):
    snapshots = db.query(LongitudinalSnapshot).filter(
```

```

LongitudinalSnapshot.user_id == user_id
).order_by(LongitudinalSnapshot.snapshot_date.asc()).all()

if len(snapshots) < 2:
    return {'message': 'Upload at least 2 PDFs to see longitudinal analysis'}

return await longitudinal_service.analyse(user_id, snapshots, db)

```

backend/services/longitudinal_service.py (new)

CODE

```

import pandas as pd
from scipy.stats import pearsonr

async def analyse(user_id, snapshots, db):
    # 1. Build per-parameter time series DataFrame
    rows = []
    for s in snapshots:
        params = json.loads(s.parameters_json)
        for name, value in params.items():
            rows.append({'date': s.snapshot_date, 'parameter': name, 'value': float(value)})
    df = pd.DataFrame(rows)

    # 2. Compute trajectory per parameter (linear regression slope)
    trajectories = {}
    for param, grp in df.groupby('parameter'):
        if len(grp) >= 2:
            x = range(len(grp))
            slope, _ = pearsonr(x, grp['value'])
            trajectories[param] = {
                'trend': 'improving' if slope < -0.1 else 'worsening' if slope > 0.1 else 'stable',
                'slope': round(slope, 3),
                'first': grp.iloc[0]['value'],
                'last': grp.iloc[-1]['value'],
                'delta': round(grp.iloc[-1]['value'] - grp.iloc[0]['value'], 2)
            }

    # 3. Send all snapshots + trajectories to GPT-4o for narrative
    summary = await run_longitudinal_agent(user_id, snapshots, trajectories)
    return {'trajectories': trajectories, 'summary': summary, 'snapshot_count': len(snapshots)}

```

backend/agents/longitudinal_agent.py (new Agent 7)

CODE

```

LONGITUDINAL_PROMPT = ""
You are a specialist health analyst reviewing {n} health reports spanning {date_range}.

Parameter trajectories:
{trajectories}

Condition history across reports:

```

```
{condition_history}

Risk score progression:
{risk_scores}

Write a 4–6 paragraph clinical summary covering:
1. Overall health trajectory (improving / declining / mixed)
2. Parameters showing sustained improvement — acknowledge positive changes
3. Parameters of concern — worsening trends requiring attention
4. New conditions that have emerged vs conditions that have resolved
5. Priority actions for the patient's next physician visit

Do not diagnose. Recommend physician consultation for all findings.
'''
```

4.5.2 Enhanced RAG for Chat (F5 extension)

The existing RAG chat already stores all PDF chunks in FAISS. Enhance it to add temporal metadata to each chunk so the LLM knows which time period a finding is from:

CODE

```
# In embedding_service.upsert_to_faiss() — add metadata
texts_with_metadata = [
    Document(
        page_content=chunk,
        metadata={'user_id': user_id, 'file_id': file_id, 'date': upload_date}
    ) for chunk in chunks
]
# FAISS with metadata uses FAISS.from_documents() instead of from_texts()

# Updated RAG prompt for temporal queries:
# 'Based on reports from {earliest_date} to {latest_date}, ...'
```

4.5.3 Frontend — LongitudinalPage.tsx

- **Timeline view:** Horizontal timeline bar showing each PDF upload date. Click to see parameter snapshot for that date.
- **Trajectory table:** Table: Parameter | First Value | Latest Value | Change | Trend (↑↓→ with colour).
- **Narrative panel:** GPT-4o summary rendered as formatted paragraphs with section headers.
- **Correlation heatmap:** Recharts RadarChart showing which parameters are correlated (e.g. glucose + HbA1c always move together).

F6	Condition-Specific Fitness & Diet Plans	3 Sprints	High
----	---	-----------	------

GOAL

Replace the generic fitness plan with deeply personalised protocols that account for detected conditions. A diabetic patient gets a fundamentally different plan than a hypertensive patient. Plans update dynamically when new PDFs are uploaded and conditions change.

4.6.1 Condition Protocol Library

Create `backend/utils/condition_protocols.py` — a static mapping of condition-specific exercise/diet rules:

CODE

```
CONDITION_PROTOCOLS = {
    'Possible Diabetes': {
        'exercise_restrictions': ['Avoid high-impact HIIT during glucose spikes'],
        'exercise_mandates': ['Post-meal walking 10–15 min', 'Resistance training 2x/week'],
        'diet_mandates': ['Low glycaemic index foods', 'Limit refined carbs to <130g/day'],
        'diet_restrictions': ['No sugary drinks', 'Limit fruit juice'],
        'monitoring': ['Check blood glucose before and after exercise'],
    },
    'Possible Anemia': {
        'exercise_restrictions': ['Avoid high-intensity exercise until Hb > 10 g/dL'],
        'exercise_mandates': ['Light yoga', 'Walking as tolerated'],
        'diet_mandates': ['Iron-rich foods: leafy greens, lentils, red meat (if not vegetarian)'],
        'diet_restrictions': ['Reduce coffee/tea around meals (inhibits iron absorption)'],
        'monitoring': ['Monitor fatigue and shortness of breath'],
    },
    'Elevated Creatinine': {
        'exercise_restrictions': ['Avoid protein loading supplements', 'No heavy compound lifts'],
        'exercise_mandates': ['Low-intensity cardio 30 min 3x/week'],
        'diet_mandates': ['Limit dietary protein to 0.6–0.8g/kg body weight'],
        'diet_restrictions': ['Avoid high-potassium foods if advised by physician'],
        'monitoring': ['Recheck creatinine in 4 weeks'],
    },
    # ... 4 more conditions from disease_agent.py
}
```

Updated FITNESS PLAN PROMPT in prompt templates.py

CODE

```
FITNESS_PLAN_PROMPT = ""
You are an expert clinical exercise physiologist and registered dietitian.

Patient profile:
- Age: {age}, Gender: {gender}, BMI: {bmi:.1f} ({bmi_category})
- Fitness level: {fitness_level}
- Risk level: {risk_level} (score: {risk_score}/100)

Detected conditions requiring protocol modification:
{conditions_with_protocols}

Each condition above has specific exercise and diet rules that MUST be followed.
```

Where protocols conflict, always apply the MORE conservative restriction.

Generate a comprehensive JSON fitness plan:

```
{{
  "workout": ["..."],    // 6-8 specific exercises with sets/reps/duration
  "diet": ["..."],       // 6-8 specific dietary recommendations
  "restrictions": ["..."], // ALL restrictions from condition protocols
  "lifestyle": ["..."],  // Sleep, stress, monitoring, physician follow-up
  "condition_notes": {{  // Per-condition special instructions
    "condition_name": "...
  }}
}}
```

backend/agents/fitness_agent.py — updated run_fitness_agent()

```
def build_condition_context(conditions: list[str]) -> str:
    lines = []
    for condition in conditions:
        protocol = CONDITION_PROTOCOLS.get(condition, {})
        if protocol:
            lines.append(f' {condition}:')
            for k, v in protocol.items():
                lines.append(f'   {k}: {v}')
    return ' '.join(lines) if lines else 'No conditions detected — general wellness plan'
```

CODE

```
async def run_fitness_agent(user_id, age, gender, bmi, risk_level,
                             risk_score, conditions):
    conditions_with_protocols = build_condition_context(conditions)
    prompt = FITNESS_PLAN_PROMPT.format(
        age=age, gender=gender, bmi=bmi,
        bmi_category=get_bmi_category(bmi),
        fitness_level=get_fitness_level(bmi, risk_level),
        risk_level=risk_level, risk_score=risk_score,
        conditions_with_protocols=conditions_with_protocols
    )
    # ... existing LLM call and JSON parsing
```

4.6.2 Plan Versioning

Store each generated plan in the new `condition_protocols` table so users can compare how their plan evolved as their conditions changed:

CODE

```
class ConditionProtocol(Base):
    __tablename__ = "condition_protocols"
    id = Column(Integer, primary_key=True)
    user_id = Column(String, ForeignKey("users.user_id"), index=True)
    generated_at = Column(DateTime, default=datetime.utcnow)
    conditions_json = Column(Text) # conditions at time of generation
```

```
risk_level    = Column(String)
plan_json     = Column(Text)   # full FitnessPlanResponse JSON
file_id       = Column(String) # which PDF triggered this version
```

4.6.3 Frontend Updates

- **FitnessPage.tsx:** Add condition pills at the top showing which conditions influenced the plan. Each pill is clickable to show the specific protocol that was applied.
- **Plan history:** Accordion at the bottom of FitnessPage showing previous plan versions with dates. User can compare old vs current plan.
- **Condition notes card:** New fifth card in the 4-card grid showing condition_notes from the plan JSON.

5 Sprint Plan — 12 Sprints / 24 Weeks

Each sprint is 2 weeks. Sprints are organised into two phases aligned with the existing project roadmap.

PHASE 1 · Foundation & Monetisation — Sprints 1–4

Sprint 1 Weeks 1–2

Goal: Google OAuth end-to-end

- Scaffold backend/routes/auth.py and JWT utilities
- Add SessionMiddleware to app.py
- Protect all 5 existing routes with get_current_user dependency
- Build LoginPage.tsx and AuthSuccess.tsx in frontend
- Add Axios Bearer token interceptor to api.ts
- Write unit tests for JWT encode/decode and auth dependency
- E2E test: full Google sign-in in browser with test account

Sprint 2 Weeks 3–4

Goal: Stripe Setup & Checkout

- Create Stripe products and price IDs in dashboard
- Build backend/routes/billing.py with checkout endpoint
- Add subscriptions table migration
- Build BillingPage.tsx with pricing cards
- Handle checkout.session.completed webhook → activate plan in DB
- Add PlanBadge component to Navbar
- Test: full checkout flow with Stripe test card 4242 4242 4242 4242

Sprint 3 Weeks 5–6

Goal: Quota Enforcement & Billing Portal

- Implement plan_guard.py with require_plan() and check_upload_quota()
- Gate /chat, /fitness/plan, and future /export.pdf routes
- Build UpgradePrompt modal component
- Add GET /billing/portal (Stripe customer portal redirect)
- Handle customer.subscription.deleted and invoice.payment_failed webhooks
- Integration tests: quota enforcement for each plan tier

Sprint 4
Weeks 7–8**Goal:** Trend Charts

- Add GET /trends/{user_id} endpoint
- Modify process_pdf() to write LongitudinalSnapshot after each upload
- Build TrendsPage.tsx with Recharts LineChart and parameter selector
- Add risk score AreaChart using snapshots data
- Implement 30/90/180/all-time filter on frontend
- Unit tests for trend endpoint aggregation logic

PHASE 2 · Intelligence & Clinical Features — Sprints 5–12**Sprint 5**
Weeks 9–10**Goal:** Medication CRUD

- Add medications and medication_logs tables
- Build all 7 medication CRUD endpoints
- Build MedicationsPage.tsx: add form + active medication cards
- Implement daily log widget (Taken/Skip/Delay buttons)
- Unit tests: medication CRUD and log endpoints

Sprint 6
Weeks 11–12**Goal:** Medication Interactions

- Embed drug_knowledge/ files into dedicated FAISS index on startup
- Build med_interaction_agent.py with LLM prompt and JSON parsing
- Build GET /medications/{user_id}/interactions endpoint
- Add adherence chart to MedicationsPage (14-day BarChart)
- Add interactions panel with severity colour coding
- Unit tests: interaction detection logic and severity classification

Sprint 7
Weeks 13–14**Goal:** Longitudinal Data Layer

- Add temporal metadata to FAISS chunk embeddings
- Update upsert_to_faiss() to use Document objects with metadata
- Verify snapshot creation on each PDF upload
- Build longitudinal_service.py with pandas trajectory analysis
- Unit tests: trajectory calculation for improving/worsening/stable cases

Sprint 8

Weeks 15–16

Goal: Longitudinal Agent & API

- Build `longitudinal_agent.py` with GPT-4o narrative generation
- Build `GET /longitudinal/{user_id}` endpoint
- Update RAG chat to include temporal metadata in context
- Unit tests: longitudinal agent with mock snapshot data
- Integration test: upload 3 PDFs and verify trajectory calculation

Sprint 9

Weeks 17–18

Goal: Longitudinal Frontend

- Build `LongitudinalPage.tsx` with timeline view
- Implement trajectory table with trend indicators
- Add narrative panel with formatted GPT-4o summary
- Add route to `Navbar` and `App.tsx`
- E2E test: longitudinal flow with test account + multiple PDFs

Sprint 10

Weeks 19–20

Goal: Condition Protocol Library

- Build `condition_protocols.py` with all 7 condition protocols
- Add `condition_protocols` table for plan versioning
- Update `FITNESS_PLAN_PROMPT` with condition context injection
- Unit tests: protocol selection for each of the 7 conditions
- Unit tests: conflict resolution (conservative rule wins)

Sprint 11

Weeks 21–22

Goal: Condition-Specific Plan Generation

- Update `run_fitness_agent()` to use condition protocols
- Store each generated plan in `condition_protocols` table
- Update `FitnessPage.tsx`: add condition pills and `condition_notes` card
- Build plan history accordion with version comparison
- Integration tests: plan generation for each condition combination

Sprint 12

Weeks 23–24

Goal: Hardening, PDF Export & Launch Prep

- Build `GET /insights/{user_id}/export.pdf` (ReportLab, Premium plan only)
- Full regression test suite across all 6 features
- Load testing: simulate 50 concurrent users on `/chat` and `/upload`

- Security review: JWT expiry, webhook signature validation, quota bypass attempts
- Update Docker Compose and .env.example with all new variables
- Write Phase 2 Azure migration notes for new services

6 Complete API Reference — New Endpoints

Method		Path	Auth	Plan	Description
GET		/auth/login	None	Any	Redirect to Google OAuth
GET		/auth/callback	None	Any	OAuth callback — issues JWT
GET		/auth/me	JWT	Any	Returns current user profile
POST		/billing/create-checkout-session	JWT	Any	Creates Stripe checkout session
POST		/billing/webhook	None	N/A	Stripe webhook handler
GET		/billing/portal	JWT	Standard+	Stripe customer portal redirect
GET		/billing/status	JWT	Any	Returns current plan + usage
GET		/trends/{user_id}	JWT	Free+	Time-series data for parameters
GET		/longitudinal/{user_id}	JWT	Standard+	Multi-report longitudinal analysis
POST		/medications/	JWT	Free+	Add medication
GET		/medications/{user_id}	JWT	Free+	List active medications
PUT		/medications/{id}	JWT	Free+	Update medication
DELETE		/medications/{id}	JWT	Free+	Deactivate medication
POST		/medications/{id}/log	JWT	Free+	Log medication event
GET		/medications/{user_id}/adherence	JWT	Free+	Adherence stats (30 days)
GET		/medications/{user_id}/interactions	JWT	Standard+	AI drug interaction check
GET		/insights/{user_id}/export.pdf	JWT	Premium	Download health report as PDF

7 Testing Strategy

7.1 New Test Files

File	Type	Coverage
tests/unit/test_jwt_utils.py	Unit	Token creation, expiry, invalid signature
tests/unit/test_plan_guard.py	Unit	Quota enforcement per plan tier
tests/unit/test_trend_aggregation.py	Unit	Time-series grouping and slope calculation
tests/unit/test_condition_protocols.py	Unit	Protocol selection for all 7 conditions
tests/unit/test_med_interactions.py	Unit	Interaction detection logic
tests/integration/test_auth_flow.py	Integration	JWT auth on all protected endpoints
tests/integration/test_billing.py	Integration	Webhook handling for all Stripe events
tests/integration/test_longitudinal.py	Integration	Multi-PDF upload → trajectory calculation
tests/integration/test_condition_plans.py	Integration	Condition → protocol → plan generation

7.2 Testing Commands

CMD	# Run all tests pytest tests/ -v --cov=backend --cov-report=html
	# Run only new feature tests pytest tests/ -k 'auth or billing or trend or longitudinal or condition' -v
	# Test inside Docker docker compose exec backend pytest tests/ -v
	# Stripe webhook tests (requires CLI) stripe trigger checkout.session.completed stripe trigger customer.subscription.deleted stripe trigger invoice.payment_failed
	# Load test (requires locust) pip install locust locust -f tests/load/locustfile.py --headless -u 50 -r 5 --run-time 60s

8 Security Checklist

Item	Status	Implementation
JWT expiry (24h)	Required	Set exp claim in create_access_token()
Stripe signature verification	Required	stripe.Webhook.construct_event() on every webhook
User isolation (JWT vs user_id)	Required	current_user['sub'] == user_id check on all routes
Quota bypass prevention	Required	Server-side check only — never trust client plan claims
OAuth state parameter	Authlib	Authlib handles CSRF protection via state param automatically
Webhook endpoint: no JWT	Required	Stripe webhook cannot send JWT — verify via signature only
SECRET_KEY rotation	Phase 2	Move to Azure Key Vault before production
Rate limiting	Phase 2	Integrate slowapi with Redis (container already running)
HTTPS enforcement	Phase 2	Azure Application Gateway terminates TLS

9 Updated File Structure

New files and directories added on top of the existing Phase 1 structure are marked with **[NEW]**:

```
backend/
├── routes/
│   ├── auth.py           [NEW] Google OAuth + JWT
│   ├── billing.py        [NEW] Stripe checkout + webhook
│   ├── trends.py         [NEW] Time-series endpoint
│   ├── longitudinal.py    [NEW] Longitudinal analysis
│   └── medications.py     [NEW] Medication CRUD
├── agents/
│   ├── med_interaction_agent.py [NEW] Drug interaction LLM
│   └── longitudinal_agent.py    [NEW] Narrative generation LLM
├── services/
│   ├── longitudinal_service.py [NEW] Trajectory analysis
│   └── med_service.py          [NEW] Medication orchestration
├── utils/
│   ├── jwt_utils.py          [NEW] Token create/decode
│   ├── auth_dep.py           [NEW] FastAPI auth dependency
│   ├── plan_guard.py          [NEW] Quota enforcement
│   └── condition_protocols.py [NEW] 7-condition protocol map
└── frontend/src/pages/
    ├── LoginPage.tsx         [NEW]
    ├── AuthSuccess.tsx       [NEW]
    └── BillingPage.tsx        [NEW]
```

```
├── TrendsPage.tsx           [NEW]
├── MedicationsPage.tsx      [NEW]
├── LongitudinalPage.tsx     [NEW]

frontend/src/components/
├── PlanBadge.tsx           [NEW]
├── UpgradePrompt.tsx       [NEW]

tests/
├── unit/test_jwt_utils.py   [NEW]
├── unit/test_plan_guard.py  [NEW]
├── unit/test_condition_protocols.py [NEW]
├── integration/test_auth_flow.py [NEW]
├── integration/test_billing.py [NEW]
├── integration/test_longitudinal.py [NEW]
```

10 Quick-Start Checklist for Developers

Use this checklist at the start of each sprint to verify readiness before writing code.

Before Sprint 1 (F1 — Google OAuth)

1. Create Google Cloud project at console.cloud.google.com
2. Enable Google Identity API and create OAuth 2.0 credentials
3. Set Authorised redirect URI to <http://localhost:8000/auth/callback>
4. Copy `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` to `config/.env`
5. Generate `APP_SECRET_KEY`: `openssl rand -hex 32`
6. Run: `pip install authlib==1.3.0 itsdangerous==2.2.0`
7. Confirm existing tests still pass: `pytest tests/ -v`

Before Sprint 2 (F2 — Stripe)

8. Create Stripe account at dashboard.stripe.com
9. Create 2 products (Standard, Premium) with monthly recurring prices
10. Copy price IDs, secret key, and webhook secret to `config/.env`
11. Install Stripe CLI for local webhook testing
12. Run: `pip install stripe==9.6.0`

Before Sprint 5 (F4 — Medications)

13. Verify `data/drug_knowledge/` directory contains `.txt` interaction files
14. Run: `pip install APScheduler==3.10.4`
15. Plan the FAISS drug knowledge index initialisation in `app.py` lifespan

Before Sprint 7 (F5 — Longitudinal)

1. Run: `pip install pandas==2.2.0 scipy==1.13.0`
2. Verify at least 2 test PDFs are available in `tests/fixtures/` for integration testing
3. Confirm `longitudinal_snapshots` table is being written on every upload

IMPORTANT

Never skip the plan guard tests before merging any feature to main.
A quota bypass vulnerability can expose premium features to free users.
Always test with Stripe test cards — never use real payment details in development.
Rotate `APP_SECRET_KEY` before any staging or production deployment.